

Guía de Migración:

NgModules → Standalone Components

Proyecto KaiOra — Ionic + Angular

Justificación del enfoque NgModules

Como estudiantes de Duoc UC, nuestro objetivo académico y profesional trasciende la mera implementación de tecnologías superficiales; buscamos comprender el funcionamiento interno y la ingeniería de los sistemas al menor detalle posible.

Si bien reconocemos que las Standalone Components son la arquitectura priorizada en las versiones actuales de Angular debido a su capacidad para reducir el código redundante (*boilerplate*), hemos optado conscientemente por cimentar este proyecto sobre la base histórica del framework: **NgModules**.

Esta decisión estratégica nos ha permitido profundizar en el corazón de Angular, dominando aspectos críticos como la resolución del contexto de compilación, el encapsulamiento de componentes y la gestión centralizada de la inyección de dependencias. Al dominar la arquitectura modular tradicional, adquirimos una perspectiva mucho más amplia y crítica del framework. Esto no solo nos capacita para entender el *porqué* de su evolución actual, sino que establece los fundamentos técnicos necesarios para ejecutar y evaluar con éxito el proceso de migración hacia Standalone descrito en este documento.

Adicionalmente, esta base nos prepara para el entorno laboral real: gran parte de los proyectos empresariales (*enterprise*) en producción hoy en día fueron construidos con NgModules. Saber migrarlos de forma segura e incremental es una habilidad concreta y valorada en el mercado laboral actual.

¿Qué son los Standalone Components?

A partir de Angular 14, el equipo de Angular introdujo los **Standalone Components** como una alternativa oficial a los NgModules. Desde Angular 17, son la arquitectura por defecto al crear nuevos proyectos.

Un componente standalone se declara con `standalone: true` en su decorador `@Component` y gestiona sus propias dependencias directamente, sin necesidad de pertenecer a un `NgModule`.

Comparativa directa

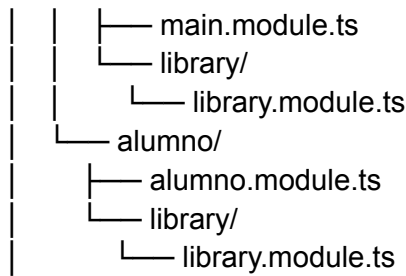
Aspecto	NgModules	Standalone
Declaración	<code>declarations: []</code> en un módulo	<code>standalone: true</code> en el componente
Importación de dependencias	En el módulo (<code>imports: []</code>)	Directamente en el componente (<code>imports: []</code>)
Boilerplate	Alto (archivo <code>.module.ts</code> por feature)	Bajo (sin archivos de módulo)
Tree-shaking	Menos granular	Más eficiente
Lazy loading	Por módulo	Por componente individual
Curva de aprendizaje	Mayor (requiere entender el sistema de módulos)	Menor para proyectos nuevos
Proyectos legacy	Predominante	En adopción progresiva

Impacto en el proyecto KaiOra

Estructura actual (NgModules)

En KaiOra, cada sección de la app tiene su propio módulo:

```
src/app/
├── app.module.ts      ← Módulo raíz
├── shared/
│   ├── shared-module.ts ← Módulo compartido (componentes reutilizables)
│   └── pages/
│       ├── admin/
│       │   └── admin.module.ts
│       └── main/      ← Módulo del profesor
```



Los componentes compartidos (navbar, modales, recipe-library) están centralizados en el **SharedModule** y se importan en cada módulo que los necesita:

```
// main.module.ts (actual)
@NgModule({
  imports: [
    CommonModule,
    IonicModule,
    SharedModule, // ← da acceso a NavBarComponent, AddUpdateRecipeComponent, etc.
    MainPageRoutingModule,
  ],
  declarations: [HomePage]
})
export class MainPageModule {}
```

Estructura después de la migración (Standalone)

Con Standalone Components, los archivos **.module.ts** desaparecen. Cada componente declara directamente lo que necesita:

```
// home.page.ts (post-migración)
@Component({
  selector: 'app-home',
  templateUrl: './home.page.html',
  styleUrls: ['./home.page.scss'],
  standalone: true,
  imports: [
    CommonModule,
    IonicModule,
    RouterModule,
    NavBarComponent, // ← importación directa
    AddUpdateRecipeComponent, // ← importación directa
    ActivateCourseModalComponent,
    NgIf,
    NgFor,
  ]
})
export class HomePage { ... }
```

Proceso de migración paso a paso

Angular provee un **esquemático oficial de migración automática**. Sin embargo, es importante entender qué hace internamente para poder corregir lo que no migre correctamente de forma automática.

Paso 1 — Prerrequisitos

Verificar versión de Angular (la migración automática requiere Angular 15+):

```
ng version
```

Si es necesario actualizar:

```
ng update @angular/core @angular/cli  
ng update @ionic/angular
```

Paso 2 — Migración automática con el esquemático oficial

Angular CLI provee un comando que convierte automáticamente todos los componentes, directivas y pipes a standalone:

```
ng generate @angular/core:standalone
```

El CLI presentará tres opciones en orden. **Ejecutarlas en este orden es crítico:**

? Choose the type of migration:

- › Convert all components, directives and pipes to standalone
- Remove unnecessary NgModules
- Bootstrap the application using standalone APIs

Opción 1 — Convertir componentes a standalone:

- Agrega **standalone: true** a cada **@Component**, **@Directive** y **@Pipe**
- Mueve las dependencias del módulo al array **imports** del componente
- Actualiza las referencias en los módulos existentes

Opción 2 — Eliminar NgModules innecesarios:

- Elimina los archivos `.module.ts` que ya no declaran nada propio
- Mantiene los módulos de routing por ahora

Opción 3 — Migrar el bootstrap:

- Reemplaza `platformBrowserDynamic().bootstrapModule(AppModule)` en `main.ts`
- Usa el nuevo `bootstrapApplication()` con providers explícitos

Paso 3 — Migración manual del bootstrap

Después del esquemático, `main.ts` debe quedar así:

```
// main.ts (post-migración)
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { provideIonicAngular } from '@ionic/angular/standalone';
import { importProvidersFrom } from '@angular/core';
import { initializeApp, provideFirebaseApp } from '@angular/fire/app';
import { provideAuth, getAuth } from '@angular/fire/auth';
import { provideFirestore, getFirestore } from '@angular/fire/firestore';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';
import { environment } from './environments/environment';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes),
    provideIonicAngular(),
    provideFirebaseApp(() => initializeApp(environment.firebaseConfig)),
    provideAuth(() => getAuth()),
    provideFirestore(() => getFirestore()),
  ]
});
```

Paso 4 — Migrar el routing

Reemplazar `app-routing.module.ts` por `app.routes.ts`:

```
// app.routes.ts (post-migración)
import { Routes } from '@angular/router';
import { AuthGuard } from './guard/auth.guard';
import { NoAuthGuard } from './guard/no-auth.guard';
import { AdminGuard } from './guard/admin.guard';
import { ProfessorGuard } from './guard/professor.guard';
import { AlumnoGuard } from './guard/alumno.guard';
```

```

export const routes: Routes = [
  {
    path: "",
    redirectTo: 'auth',
    pathMatch: 'full'
  },
  {
    path: 'auth',
    loadComponent: () => import('./pages/auth/auth.page').then(m => m.AuthPage),
    canActivate: [NoAuthGuard]
  },
  {
    path: 'admin',
    canActivate: [AuthGuard, AdminGuard],
    loadChildren: () => import('./pages/admin/admin.routes').then(m => m.adminRoutes)
  },
  {
    path: 'main',
    canActivate: [AuthGuard, ProfessorGuard],
    loadChildren: () => import('./pages/main/main.routes').then(m => m.mainRoutes)
  },
  {
    path: 'alumno',
    canActivate: [AuthGuard, AlumnoGuard],
    loadChildren: () => import('./pages/alumno/alumno.routes').then(m => m.alumnoRoutes)
  },
];

```

Cada sección tiene su propio archivo de rutas, por ejemplo:

```

// pages/main/main.routes.ts
import { Routes } from '@angular/router';

export const mainRoutes: Routes = [
  {
    path: "",
    redirectTo: 'home',
    pathMatch: 'full'
  },
  {
    path: 'home',
    loadComponent: () => import('./home/home.page').then(m => m.HomePage)
  },
  {
    path: 'library',
    loadComponent: () => import('./library/library.page').then(m => m.LibraryPage)
  }
];

```

```
},  
];
```

Paso 5 — Actualizar los guards

Los guards pasan de **CanActivate** (clase) a funciones puras:

```
// guard/auth.guard.ts (post-migración)  
import { inject } from '@angular/core';  
import { CanActivateFn } from '@angular/router';  
import { Utils } from '../services/utils';  
  
export const AuthGuard: CanActivateFn = () => {  
  const utilsSvc = inject(Utils);  
  // ... lógica igual que antes  
};
```

Paso 6 — Eliminar SharedModule

El **SharedModule** pierde su razón de existir. Cada componente que usaba **SharedModule** ahora importa directamente lo que necesita:

```
// Antes (con NgModules):  
// En el módulo se importaba SharedModule y eso daba acceso a NavBarComponent  
  
// Después (Standalone):  
@Component({  
  standalone: true,  
  imports: [NavBarComponent, RecipeLibraryComponent, IonicModule, CommonModule]  
})  
export class LibraryPage { }
```

Puntos críticos específicos de KaiOra

SharedModule → importaciones directas

Todos los componentes que actualmente se registran en **SharedModule** deben convertirse a standalone y ser importados directamente donde se usen:

Componente actual en SharedModule	Acción post-migración
-----------------------------------	-----------------------

<code>NavBarComponent</code>	<code>standalone: true</code> → importar en cada page
<code>AddUpdateRecipeComponent</code>	<code>standalone: true</code> → importar donde se abre como modal
<code>ActivateCourseModalComponent</code>	<code>standalone: true</code> → importar donde se abre como modal
<code>RecipeViewModalComponent</code>	<code>standalone: true</code> → importar en <code>RecipeLibraryComponent</code>
<code>RecipeLibraryComponent</code>	<code>standalone: true</code> → importar en páginas de librería

Modales con `ModalController`

Las modales de Ionic presentadas con `ModalController` y `presentModal()` de `Utils` **requieren que el componente modal sea standalone** para que Ionic pueda instanciarlos dinámicamente. En la versión actual con `NgModules`, funcionan porque están declarados en el `SharedModule`. Post-migración:

```
// El componente modal debe ser standalone
@Component({
  standalone: true,
  imports: [IonicModule, CommonModule, ReactiveFormsModule, NgFor, NgIf],
  ...
})
export class AddUpdateRecipeComponent { }
```

AngularFire

Con el bootstrap standalone, los providers de Firebase se registran en `main.ts` (como se mostró en el Paso 3) en lugar de en `AppModule`. Verificar que no queden importaciones de `AngularFireModule` ni `AngularFirestoreModule` del SDK legacy.

Beneficios concretos post-migración en KaiOra

1. **Eliminación de 8+ archivos `.module.ts`** — menos archivos que mantener y sincronizar
2. **Lazy loading por componente** — cada modal o página se carga solo cuando se necesita, mejorando el tiempo de carga inicial de la app
3. **Tree-shaking más eficiente** — el build final incluye solo el código realmente utilizado

4. **Alineación con el estándar actual** — facilita el uso de las últimas APIs de Angular (signals, `@if`, `@for`) sin adaptadores
 5. **Menor acoplamiento** — cada componente es autosuficiente y más fácil de testear de forma aislada
-

Riesgos y consideraciones

- **El esquemático no es perfecto** — en proyectos con lógica compleja (como modales dinámicas y guards personalizados), revisar manualmente lo que genera
- **Ionic standalone** — algunas APIs de Ionic tienen imports específicos para standalone (ej. `IonButton`, `IonContent` en lugar de `IonicModule` completo). Revisar la documentación de [@ionic/angular/standalone](#)
- **Probar la app completa** — después de la migración, recorrer todos los flujos (login admin, crear curso, login profesor, crear ficha, login alumno, ver ficha) para detectar dependencias que el esquemático no haya migrado correctamente